

Machine Learning Solutions to Stochastic Reaction Networks

Yoey Tolboom

Supervisor:
Dr. Chiheb Ben Hammouda



Department of Mathematics
Utrecht University
June 2026

This thesis is presented for the degree of Bachelor of science

Abstract. Stochastic reaction networks (SRNs) are often applied to model stochastic processes in physics, chemistry and biology. In SRNs, chemical species evolve over time by a set of reactions. The evolution of the joint probability distribution of the species counts over time can be determined by solving the chemical master equation (CME). In a 2023 paper called "Neural-network solutions to stochastic reaction networks" [2], the authors proposed a variational autoregressive network (VAN) for solving the CME. Their method involves training a VAN for every time step of a discretized time interval. Their method scales very well with the number of species in the SRN, even though the size of the state space grows exponentially with the amount of species. Nonetheless, there are several issues with their approach which involve error-propagation, large number of parameters and long training times. This is why we propose a slightly altered approach to the VAN; the globally time-conditioned autoregressive network. In this approach we train a single model to capture the entire joint probability distribution over time. The model has been found to correctly fit the entire joint probability distribution over time of some simpler SRNs, like the birth-death process and the linear signaling cascade. In these cases it requires less time to train than the VAN, using fewer parameters. However, for the gene expression SRN, the model has been found to completely fail. Furthermore, the training time has been found to not scale well with the amount of species in the SRN.

Contents

1	Introduction	3
2	Background and Mathematical Framework	4
2.1	Reactions	4
2.2	Chemical Master Equation	4
2.3	Time-Continuous Markov Chains	6
2.3.1	General Case	6
2.3.2	Stochastic Reaction Networks as Time-Continuous Markov Chains	7
2.4	Deriving the Loss Function of the Variational Autoregressive Network	8
3	Methodology	10
3.1	Basic Time-Conditioned Model	10
3.1.1	Forward Pass	10
3.1.2	Training	11
3.1.3	Taking Samples	12
3.1.4	Computing $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$	13
3.1.5	Selecting a Value for δt	13
3.1.6	Baseline	14
3.2	Time-Conditioned Model with Masked Attention Mechanism	14
3.2.1	Masked Attention	15
3.2.2	Forward Pass	15
4	Numerical Experiments and Results	16
4.1	The Birth-Death Process	17
4.2	Gene Expression	19
4.3	Linear Signaling Cascade	21
4.4	Time Embedding Tests	23
4.5	Runtime Scaling	25
5	Discussion	26
6	Conclusion	26
7	Code Availability	26

1 Introduction

Often in nature we find dynamic systems of things that tightly interact with each other. For example we find ecosystems, where there are many different species of organisms competing for resources in the same area. Or we could have a mixture of chemicals, of which some can undergo chemical reactions. The most straightforward way of modeling such dynamics is done with differential equations in chemical reaction networks, which works well when we have a relatively large number of 'things'. The idea behind stochastic reaction networks (SRNs) is modeling such dynamic systems for a potentially small number of things. Therefore to get a model that gives us an accurate summary for the dynamics, it helps to involve stochastic modeling. Instead of a chemical reaction network we then have a stochastic reaction network. In this thesis, we will be going over the background that is necessary for our specific research question in section 2. And we assume that the reader is familiar with university-level mathematics on probability theory, calculus and linear algebra. Our research focuses on solving the joint probability distribution associated with any SRN using machine learning methods. This has already been done before in a 2023 paper, where they used a variational autoregressive network (VAN) [2] (we will sometimes refer to this paper as the "VAN paper"). Specifically, in this thesis we investigate whether a single globally time-conditioned autoregressive network can approximate the time-dependent joint probability distribution of SRNs. We do this as an alternative to training a VAN model for every time step of a discretized time interval, which is the method they employed in the VAN paper. The globally time-conditioned model that we investigate has some key differences to the VAN approach. First of all, it is time-conditioned, meaning that the model takes in time as an argument. Secondly, it has a single shared set of parameters for the entire time interval that we train on. This is in contrast to the VAN approach, which uses a different set of parameters for every time step. We discuss the VAN approach before presenting our own method in section 3, which is ultimately based on the VAN approach. Then in section 4, we will go over the results of our numerical investigations of our method, where we tested on benchmark SRNs. Finally, we will end by discussing the results and possible future avenues for research in section 5.

2 Background and Mathematical Framework

We use the following conventions:

$$\begin{aligned}\mathbb{N} &:= \{1, 2, 3, \dots\} \\ \mathbb{N}_0 &:= \{0, 1, 2, \dots\}\end{aligned}$$

2.1 Reactions

We will introduce a mathematical notion of what it means for things to interact, this will be the idea behind a *reaction*. For example 1 molecule of both chemical A and B might react and create a new molecule of chemical C , which is constituted of 1 molecule of chemical A and 1 molecule of chemical B . In this case we consider 1 molecule of both chemical A and B to be destroyed in the process. Or a bacterium of the species S might "react" and replicate itself. We can write the chemical reaction of the first example as:

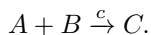


And similarly we can write the act of the bacterium replicating itself as such:



We would call the reactants in these examples *species*. Sometimes this refers to an actual species in biology, as is the case for the bacterium S . When modeling an environment with such species and some reactions between species, the main objective is knowing the *population* of all the species at some time $t > 0$, given an initial population count for each species.

Notice that how often a reaction takes place will depend on the population count of a species. Or in other words, it will depend on how many reactants there are in that environment. For example, in a petri dish with well-fed bacteria, we will assume that the amount of bacteria that will replicate themselves according to eq. (2.2) will be proportional to the amount of bacteria there are. Furthermore, in the cases that we have described there is some unpredictability. We cannot predict exactly when two chemicals will collide and react, just as we cannot predict exactly when a bacterium might replicate itself. To model this we can add a constant called the *reaction rate*. We define it such that the amount of reactions we expect to take place in a unit of time is proportional to it. The reaction rate is tied to the specific reaction, for example if we know that the reaction rate for eq. (2.16) is $c \in \mathbb{R}_{\geq 0}$, then we can write the reaction like so:



In general we have the species $S_1, \dots, S_M$ and $K \in \mathbb{N}$ reactions:

$$\sum_{i=1}^M r_{k,i} S_i \xrightarrow{c_k} \sum_{i=1}^M p_{k,i} S_i. \quad (2.3)$$

Where $M \in \mathbb{N}$ and $1 \leq k \leq K$. The constants $r_{k,i}$ denote the amount of the species S_i that will react for the k 'th reaction to take place. Furthermore the constants $p_{k,i}$ denote the amount of the species S_i that will be produced by the k 'th reaction. Finally c_k denotes the reaction rate of the k 'th reaction.

2.2 Chemical Master Equation

Now that we have a more mathematical notion of reactions, we can try to actually solve the problem at hand. We will assume that the counts $n_1, \dots, n_M \in \mathbb{N}_0$ of each of our species $S_1, \dots, S_M$ are discrete. Because of this we will try to determine the probability of having the species counts: $n_1, \dots, n_M$ at some point in time $t > 0$. For ease of notation we will write $n_1, \dots, n_M$ as the M -dimensional vector $\mathbf{n} = (n_1 \ \dots \ n_M)^\top$, we call this the *state* of the system, and we call the space of all possible states $\mathcal{S} := \mathbb{N}_0^M$ the *state space*. When computing with SRNs, we truncate the state space to $\{0, \dots, N-1\}^M$

for some $N \in \mathbb{N}$. Now we can try to derive a formula for $\mathbb{P}(\mathbf{n}, t)$, that is: the probability that we are in state $\mathbf{n}$ at time t .

We will now assume *stochastic mass-action*. Meaning that we define the *propensity* $a_k(\mathbf{n})$ for reaction k , where: $1 \leq k \leq K$ as follows:

$$a_k(\mathbf{n}) := c_k \prod_{i=1}^M \binom{n_i}{r_{k,i}}$$

Where $c_k > 0$ is the stochastic rate constant (reaction rate) and $\binom{n_i}{r_{k,i}}$ counts the number of ways to select the $r_{k,i}$ reactant molecules of species i . The propensity represents the rate at which the reaction k occurs in an infinitesimally small time interval at the state $\mathbf{n}$. Thus, the higher the propensity for reaction k , the more we expect this reaction to take place. In the example of the bacteria, we would have: $a_1(n) = cn$. We only had one reaction in the example, and thus this would be the only propensity. We use the convention that whenever $n_i < r_{k,i}$ that:

$$\binom{n_i}{r_{k,i}} = 0. \quad (2.4)$$

This convention counts the number of possible combinations of reactant molecules available for reaction k . For first-order reactions and reactions involving distinct species only, this reduces to the usual power-law mass-action¹ expression up to the choice of rate constant. To explain the rationale behind this, we can take another concrete example. Say we have two different chemicals X and Y suspended in a solution. Suppose there are n_1 particles of the X chemical in the solution and n_2 particles of the Y chemical. Assume some reaction can take place between these chemicals where it needs one particle of X and two particles of Y to take place. Say the reaction can only happen when these particles collide. Then for a single reaction to take place one of our X particles will have to collide with two of our Y particles. So we first of all expect the reaction to take place more often when we have more particles of either species. Furthermore, as we need two Y particles, we need any combination of our n_2 particles to collide with the other X particle, this is where the $\binom{n_i}{r_{k,i}}$ product comes from.

Say that at some time $t + \delta t$ we have $n \in \mathbb{N}_0$ bacteria of the species S from the reaction eq. (2.2), where we assume it has reaction rate k . If we consider increasingly smaller time steps δt , that the reactions we expect to take place within the time interval $[t, t + \delta t]$ will tend towards 0. Importantly, the probability of reaction k taking place in $[t, t + \delta t]$ will be:

$$\mathbb{P}(\text{reaction } k \text{ occurs in } [t, t + \delta t] \mid \text{the state at time } t \text{ is } \mathbf{n}) = a_k(\mathbf{n})\delta t + \mathcal{O}((\delta t)^2).$$

However, the probability of two or more reactions taking place in $[t, t + \delta t]$ will be of order $\mathcal{O}((\delta t)^2)$:

$$\mathbb{P}(\text{two or more reactions occur in } [t, t + \delta t] \mid \text{the state at time } t \text{ is } \mathbf{n}) = \mathcal{O}((\delta t)^2).$$

We also know that if we had n bacteria at time $t + \delta t$, that we had either $n - 1$ or n at time t if no more than 1 reaction took place. Because the reaction eq. (2.2) tells us that one reaction taking place will increase the number of the bacteria S by 1. If no reaction took place then the species count n will obviously remain the same. Taking eq. (2.2) as an example, it follows from our earlier definitions and assumptions that:

$$\mathbb{P}(n, t + \delta t) = \mathbb{P}(n - 1, t)c(n - 1)\delta t + \mathbb{P}(n, t)(1 - cn\delta t) + \mathcal{O}((\delta t)^2). \quad (2.5)$$

Where the first term of the right side represents the situation where we had $n - 1$ bacteria at time t and the reaction did take place within the time interval $[t, t + \delta t]$. And the second term of the right

¹For power-law mass-action the propensities are defined as follows: $a_k(\mathbf{n}) := c_k \prod_{i=1}^M n_i^{r_{k,i}}$

side represents the situation where we already had n bacteria at time t , and no reaction took place within the time interval $[t, t + \delta t]$. We can rewrite eq. (2.5) to arrive at:

$$\frac{\mathbb{P}(n, t + \delta t) - \mathbb{P}(n, t)}{\delta t} = c(n-1)\mathbb{P}(n-1, t) - cn\mathbb{P}(n, t) + \mathcal{O}(\delta t).$$

The left side of this equation should look very familiar, as we let $\delta t \rightarrow 0^+$, it converges to the derivative with respect to t . And thus we derive the following ordinary differential equation:

$$\partial_t \mathbb{P}(n, t) = c(n-1)\mathbb{P}(n-1, t) - cn\mathbb{P}(n, t). \quad (2.6)$$

This is the *chemical master equation*, which is relatively simple for this example. Note that there are some boundary cases where this equation is ill-defined, for example if we take $n = 0$ in eq. (2.6) then $n-1 = -1 \notin \mathbb{N}_0$, and will thus be outside of our state space. In general, in any state outside of our state space, it follows from eq. (2.4) that the propensity for every reaction is 0. Because if there is some species S_i for which: $n_i \notin \mathbb{N}_0$, then: $n_i < 0 \leq r_{k,i}$ for any reaction k , and so: $\binom{n_i}{r_{k,i}} = 0$. Similarly, for any state $\mathbf{n} \notin \mathcal{S}$ and any $t \geq 0$, we use the convention that:

$$\mathbb{P}(\mathbf{n}, t) = 0.$$

For general stochastic reaction networks it gets a bit more complicated, but the general idea is the same. Say we have the species $S_1, \dots, S_M$, the state space $\mathbf{n}$ and we have K reactions like in eq. (2.3). Then the chemical master equation will be:

$$\partial_t \mathbb{P}(\mathbf{n}, t) = \sum_{k=1}^K [a_k(\mathbf{n} - \nu_k)\mathbb{P}(\mathbf{n} - \nu_k, t) - a_k(\mathbf{n})\mathbb{P}(\mathbf{n}, t)]. \quad (2.7)$$

Where ν_k is the stoichiometric vector belonging to the k 'th reaction. This is defined as: $\nu_k := \mathbf{p}_k - \mathbf{r}_k \in \mathbb{Z}^M$, where: $\mathbf{r}_k := (r_{k,1}, \dots, r_{k,M})^\top$ and $\mathbf{p}_k := (p_{k,1}, \dots, p_{k,M})^\top$. The k 'th stoichiometric vector is therefore simply the vector of the net changes to the species counts that happen whenever reaction k happens.

2.3 Time-Continuous Markov Chains

We will now step back from stochastic reactions networks and look at stochastic processes more generally. Specifically, we will look at the general case for time-continuous Markov chains in section 2.3.1. Then we will see that this theory also applies to stochastic reactions networks and discuss the ramifications in section 2.3.2.

2.3.1 General Case

Recall that a continuous time stochastic process $\mathbf{X} = \{X(t) \mid t \geq 0\}$ on a countable state space $\mathcal{S}$ is a continuous-time Markov chain if the Markov-property holds. That is: For any finite set of indices $0 \leq t_1 \leq \dots \leq t_n < s < t$ and corresponding states: $i_1, \dots, i_n, i, j \in \mathcal{S}$ such that:

$$\mathbb{P}(X(t_1) = i_1, \dots, X(t_n) = i_n, X(s) = i) > 0.$$

We have:

$$\mathbb{P}(X(t) = j \mid X(s) = i, X(t_n) = i_n, \dots, X(t_1) = i_1) = \mathbb{P}(X(t) = j \mid X(s) = i).$$

Which means that given we are in state i at time s , the probability of transitioning to the state j at time t is independent of the states we were in before time s . If additionally we have for all $0 \leq s \leq t$ and $i, j \in \mathcal{S}$ that $\mathbb{P}(X(t) = j \mid X(s) = i)$ depends only on $t - s$, then we call $\mathbf{X}$ *homogeneous*. We say that $\mathbf{X}$ has stationary transition probabilities.

For $i, j \in \mathcal{S}$ and $t \geq 0$, we call:

$$p_{ij}(t) := \mathbb{P}(X(t) = i \mid X(0) = j) \quad (2.8)$$

the *transition function*² of $\mathbf{X}$. We can also put this into matrix notation $\mathbb{T}(t)$, with $p_{ij}(t)$ being the element on the i 'th row and j 'th column.

If $\mathbf{X}$ is a time-continuous homogeneous Markov chain then:

$$p_{ij}(t+s) = \sum_{k \in \mathcal{S}} p_{ik}(t)p_{kj}(s).$$

In other words, the transition probability from state j to i is equal to the sum over all $k \in \mathcal{S}$ of the transition probabilities from j to i with k as an intermediate state. When we write this in matrix notation, notice that:

$$\mathbb{T}(t+s) = \mathbb{T}(t)\mathbb{T}(s). \quad (2.9)$$

From this commutativity follows:

$$\mathbb{T}(t+s) = \mathbb{T}(t)\mathbb{T}(s) = \mathbb{T}(s)\mathbb{T}(t) = \mathbb{T}(s+t).$$

Furthermore, assuming we can take the derivative with respect to s in eq. (2.9) and evaluate at $s = 0$, we get:

$$\dot{\mathbb{T}}(t) = \dot{\mathbb{T}}(0)\mathbb{T}(t). \quad (2.10)$$

Also notice that: $\mathbb{T}(0) = I$, as: $p_{ij}(0) = 0$ whenever $i \neq j$ and $p_{ij}(0) = 1$ whenever $i = j$. This should make intuitive sense, as we cannot transition to another state in no amount of time. This, together with eq. (2.10), then gives us an ODE which we know the solution for:

$$\mathbb{T}(t) = e^{\dot{\mathbb{T}}(0)t}. \quad (2.11)$$

2.3.2 Stochastic Reaction Networks as Time-Continuous Markov Chains

Notice that for any stochastic reaction network, the state space $\mathbb{N}_0^M$ is countable and our time variable $t \geq 0$ is continuous. Also the way we defined reaction dynamics ensures that transitioning to another state is dependent on the previous state of the system, but not on all the previous states the system was in before that. Meaning that the Markov property holds. We implicitly assumed that reaction dynamics are time homogeneous: for any given reaction, the chance that that reaction takes place only depends on the current state and the interval of time $[t, t + \delta t]$. In a sense, the reactions do not have a 'memory', it does not matter at what time we look, the chance of the reaction taking place within the same length interval of time will always be the same given we are in the same state at both times. This is exactly what it means for a continuous-time Markov chain to be homogeneous. So homogeneous continuous-time Markov chains describe the dynamics of stochastic reaction networks perfectly.

Knowing this, we can uncover more properties of stochastic reactions networks. Firstly, we should rewrite the chemical master equation eq. (2.7) in a more general way:

$$\partial_t \mathbb{P}(\mathbf{n}, t) = \sum_{\mathbf{n}' \neq \mathbf{n}} [W_{\mathbf{n}' \rightarrow \mathbf{n}} \mathbb{P}(\mathbf{n}', t)] - r_{\mathbf{n}} \mathbb{P}(\mathbf{n}, t). \quad (2.12)$$

Where $W_{\mathbf{n} \rightarrow \mathbf{n}'}$ is the *transition rate* from state $\mathbf{n}$ to state $\mathbf{n}'$. Notice that this is not much different than our earlier notation using the propensities, the transition rate is simply the sum of the propensities of the reactions that would transition the state $\mathbf{n}$ to the state $\mathbf{n}'$. In the case that no reaction would do

²Note that this definition is different than the usual definition where the roles of i and j are reversed: $p_{ij}(t) := \mathbb{P}(X(t) = j \mid X(0) = i)$. We do this for cleaner notation, while the idea that we try to convey remains the same.

this, the transition rate is simply 0. We call $r_{\mathbf{n}}$ the escape rate from the state $\mathbf{n}$, this is defined as:

$$r_{\mathbf{n}} = \sum_{\mathbf{n}' \neq \mathbf{n}} W_{\mathbf{n} \rightarrow \mathbf{n}'} = \sum_{k=1}^K a_k(\mathbf{n}). \quad (2.13)$$

Which are just the transition rates from all the ways we can 'escape' the state $\mathbf{n}$ summed up. Note that we can further rewrite eq. (2.12) as such:

$$\partial_t \mathbb{P}(t) = \mathbb{W} \mathbb{P}(t). \quad (2.14)$$

Where $\mathbb{P}(t)$ is the vector: $(\mathbb{P}(\mathbf{n}, t))_{\mathbf{n} \in \mathbb{N}_0^M}^T$. And $\mathbb{W}$ is defined as:

$$\mathbb{W}_{\mathbf{n}, \mathbf{n}'} := \begin{cases} W_{\mathbf{n}' \rightarrow \mathbf{n}} & \text{if } \mathbf{n} \neq \mathbf{n}' \\ -r_{\mathbf{n}} & \text{if } \mathbf{n} = \mathbf{n}' \end{cases}.$$

Then recall that $\mathbb{T}(t)$ was the matrix of transition functions, see eq. (2.8). Where:

$$\mathbb{P}(t) = \left(\sum_{\mathbf{n}'} \mathbb{P}(X(t) = \mathbf{n} \mid X(0) = \mathbf{n}') \mathbb{P}(X(0) = \mathbf{n}') \right)_{\mathbf{n} \in \mathbb{N}_0^M}^T = \left(\sum_{\mathbf{n}'} p_{\mathbf{n}', \mathbf{n}}(t) \mathbb{P}(X(0) = \mathbf{n}') \right)_{\mathbf{n} \in \mathbb{N}_0^M}^T = \mathbb{T}(t) \mathbb{P}(0)$$

by the law of total probability. Using this and eq. (2.14), we obtain:

$$\partial_t \mathbb{P}(t) = \mathbb{W} \mathbb{P}(t) = \mathbb{W} \mathbb{T}(t) \mathbb{P}(0) = \partial_t \mathbb{T}(t) \mathbb{P}(0) = \partial_t (\mathbb{T}(t) \mathbb{P}(0))$$

for any initial condition $\mathbb{P}(0)$. And so:

$$\partial_t \mathbb{T}(t) = \mathbb{W} \mathbb{T}(t).$$

And we already established that: $\mathbb{T}(0) = I$. So solving this ODE brings us to the following equation:

$$\mathbb{T}(t) = e^{\mathbb{W}t}. \quad (2.15)$$

Notice how this is of the form we found earlier in eq. (2.11).

2.4 Deriving the Loss Function of the Variational Autoregressive Network

The Kullback–Leibler divergence can be seen of as a distance measure function for probability distributions. Let P, Q be discrete probability distribution on the sample space $\mathcal{X}$, then the Kullback–Leibler divergence is defined by:

$$D_{KL}(P \mid Q) = \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right). \quad (2.16)$$

This is just the expected value when drawing $x \sim P$ of the logarithmic difference of P and Q :

$$\begin{aligned} D_{KL}(P \mid Q) &= \sum_{x \in \mathcal{X}} P(x) \log\left(\frac{P(x)}{Q(x)}\right) \\ &= \sum_{x \in \mathcal{X}} (\log(P(x)) - \log(Q(x))) P(x) \\ &= \mathbb{E}_{x \sim P}(\log(P) - \log(Q)). \end{aligned} \quad (2.17)$$

A lot of our work will be based directly on the variational autoregressive network (VAN) proposed in a 2023 paper [2]. Now to derive some formula for the loss function, we want the VAN to learn

the next time step from some time t , where δt will be our time step. So let $\hat{P}_t^{\theta_t}$ be the learned joint probability distribution of the VAN with parameters θ_t at time t . The model at time $t + \delta t$ is trained to match one approximate CME step applied to the model at time t , where the size of the step is δt . For this we can simply left-multiply the already trained model at time t with the first order approximation of the transition matrix $\mathbb{T} \approx I + \delta t \mathbb{W}$, using eq. (2.15). Therefore, we want to minimize the following loss function:

$$\mathcal{L} = D_{KL}(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}} \mid \mathbb{T} \hat{P}_t^{\theta_t}).$$

Where $\mathbb{T} \hat{P}_t^{\theta_t}$ acts as our target probability distribution that we want the VAN to learn at time $t + \delta t$. Like earlier we can rewrite the KL-divergence like so:

$$\mathcal{L} = \mathbb{E}_{\mathbf{s} \sim \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}} (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log(\mathbb{T} \hat{P}_t^{\theta_t}(\mathbf{s}))).$$

We approximate this by taking samples $\mathbf{s}$ of the species counts from the distribution parameterized by the VAN $\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}$. Note that only $\mathcal{O}(K)$ multiplications are needed to calculate $(\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s})$, as we know that from each sample, there are only K possible states which could have transitioned to it within the time interval δt , as there are K reactions. This is later expanded upon for the time-conditioned model that we propose in section 3.1.4, but the same logic applies here. We will update the parameters of the VAN by gradient descent, the gradient is:

$$\begin{aligned} & \nabla_{\theta_{t+\delta t}} \mathcal{L} \tag{2.18} \\ &= \nabla_{\theta_{t+\delta t}} \sum_{\mathbf{s}} (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) \\ &= \sum_{\mathbf{s}} \nabla_{\theta_{t+\delta t}} \left((\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) \right) \\ &= \sum_{\mathbf{s}} \nabla_{\theta_{t+\delta t}} \log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) + \sum_{\mathbf{s}} (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \nabla_{\theta_{t+\delta t}} \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) \\ &= \nabla_{\theta_{t+\delta t}} \sum_{\mathbf{s}} \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) + \sum_{\mathbf{s}} (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \frac{\nabla_{\theta_{t+\delta t}} \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})}{\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})} \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) \\ &= \nabla_{\theta_{t+\delta t}} 1 + \sum_{\mathbf{s}} \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s}) (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \nabla_{\theta_{t+\delta t}} \log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) \\ &= \mathbb{E}_{\mathbf{s} \sim \hat{P}_{t+\delta t}^{\theta_{t+\delta t}}} \left(\nabla_{\theta_{t+\delta t}} \log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) (\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))) \right). \end{aligned}$$

This last expression is a REINFORCE-type gradient estimator [4]. Where our policy is $\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})$ and our reward is: $\log(\hat{P}_{t+\delta t}^{\theta_{t+\delta t}}(\mathbf{s})) - \log((\mathbb{T} \hat{P}_t^{\theta_t})(\mathbf{s}))$.

3 Methodology

The VAN-based approach [2] essentially trains a model at every timestep, where the step is δt and the total time T . The method clearly works given their results, and although the method is clearly faster than naive marginal distribution estimators using trajectory simulations, it still takes hours to run for some more complex SRNs. This most likely can be attributed to the large number of parameters in this model; one must train a model for approximately $\frac{T}{\delta t}$ time steps. Furthermore, it only exploits the inherent similarities between P_t and $P_{t+\delta t}$ for every time step, it does not learn a shared space-time representation. Because the model for time $t + \delta t$ is trained using the model from the previous time step, the model at time t . This also means that we cannot take advantage of parallelism in this aspect; we need to train the model of the previous time steps to train the next. Their approach could also lead to error-propagation; the learned model of the previous time step is used to train the model for the next time step. If there is some error in the previous model, the next model will train using this flawed model. Another more obvious drawback is that the learned joint probability distribution can not be evaluated at any time, instead we only have access to that of every time step. We therefore present a new approach. We will take the time into the model as a parameter. This requires time-scheduling, and some other changes that will be discussed in this section.

3.1 Basic Time-Conditioned Model

The first model we propose will serve as a baseline for the more complex model to come. It is a multilayer perceptron (MLP) with a *softmax* output layer. For a vector $\mathbf{z} \in \mathbb{R}^n$, the softmax function is defined by:

$$\text{Softmax}(\mathbf{z}) := \frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}}.$$

We will keep the autoregressive factorization from the VAN, only slightly altering it to take the time t into account:

$$\hat{P}_\theta(n, t) = \prod_{i=1}^M \hat{P}_\theta(n_i \mid n_1, \dots, n_{i-1}, t).$$

This is therefore a *globally time-conditioned autoregressive network*, instead of the variational autoregressive network (VAN) that we are basing this model on.

3.1.1 Forward Pass

Let N be the truncation count for each species. Let M again be the amount of species in our SRN. For $1 \leq i \leq M$ define the input token:

$$x_i := E(i, n_{<i}, \psi(t), \eta_i) \in \mathbb{R}^{d_x}. \quad (3.1)$$

Where E is an embedding map, $\psi : \mathbb{R} \rightarrow \mathbb{R}^{d_t}$ is a learnable time embedding (except when explicitly stated otherwise) and η_i is extra structural information for species i . We also define:

$$n_{<i} := (n_1, n_2, \dots, n_{i-1})^\top$$

As all x_i need to be of the same shape in the code, we pad out any left-over unset values in x_i with -1 . As of now the embedding map E has not been tested beyond it simply being the identity map, which it is set to by default. The learnable time embedding is a two-layer MLP with linear output layer. The time t is first projected to the feature vector: $(t, \log(t + \epsilon_1))^\top$, where $\epsilon_1 := 1^{-3}$. Then that feature vector is processed through the MLP to produce the vector $\psi(t) \in \mathbb{R}^{d_t}$. By default we use the following structural information:

$$\eta_i := (r_{1,i}, \dots, r_{K,i}, p_{1,i}, \dots, p_{K,i}, c_1, \dots, c_K)^\top.$$

That is, the structural information consists of all the reactant and product counts that relate to species i of all reactions in addition to the reaction rates of all reactions. Note how token i only contains the information of species counts from species that get a lower index than i , but not any other species counts. This is because the other species counts should not be known to the model when trying to predict the conditional probability for species i . From this, for each $1 \leq i \leq M$, x_i will be fed into an MLP. The MLP has 1 hidden layer with $d_h \in \mathbb{N}$ nodes and tanh activation function unless stated otherwise. The MLP will produce the logits $\hat{P}_\theta(n_i = m \mid n_{<i}, t)$ for all $0 \leq m < N$. The softmax activation function of the output layer will preserve total probability adding to 1. After processing x_i for all i we then produce: $\hat{P}_\theta(n, t) = \prod_{i=1}^M \hat{P}_\theta(n_i \mid n_{<i}, t)$. The following is a simple diagram explaining the employed machine learning pipeline for the basic time-conditioned model:

$$(i, n_{<i}, \psi(t), \eta_i) \rightarrow x_i \rightarrow \text{MLP} \rightarrow \text{softmax} \rightarrow \hat{P}_\theta(n_i \mid n_{<i}, t).$$

3.1.2 Training

The loss function for this model is the cross entropy or (KL-divergence) between the produced probability distribution at time $t + \delta t$ (which is $\hat{P}_\theta(\cdot, t + \delta t)$), and the target distribution ($\mathbb{T}\hat{P}_{\theta^-}(\cdot, t)$). Where $\mathbb{T} = e^{\delta t \mathbb{W}}$, as derived in eq. (2.15). Like in the VAN paper [2] we approximate $\mathbb{T}$ by first order approximation: $\mathbb{T} \approx I + \delta t \mathbb{W}$. Note that the target is not dependent on θ . Instead, we copy θ into the frozen copy θ^- every Q epochs, where Q is a hyperparameter of the model. We always set $Q = 1$ unless stated otherwise. This is to prevent the target from moving towards the predicted distribution. In summary, we minimize the following loss function:

$$\mathcal{L} = D_{KL}(\hat{P}_\theta(\cdot, t + \delta t) \mid \mathbb{T}\hat{P}_{\theta^-}(\cdot, t)).$$

As discussed earlier in eq. (2.17) this can be approximated by taking samples from $s \sim \hat{P}_\theta(\cdot, t + \delta t)$ in this case. Let B be the number of samples we take, and let $\mathbf{s}^{(b)}$ be the b 'th sample where $1 \leq b \leq B$, then:

$$\mathcal{L} \approx \frac{1}{B} \sum_{b=1}^B [\log(\hat{P}_\theta(\mathbf{s}^{(b)}, t + \delta t)) - \log((\mathbb{T}\hat{P}_{\theta^-}(\cdot, t))(\mathbf{s}^{(b)}))].$$

The parameters of the model are updated by gradient descent every epoch.

As the model takes in time as an argument, there is a schedule for which time to train on. The method we employ is taking t uniformly randomly from $[0, \frac{(T-\delta t)e}{e_{total}}]$ each epoch. Where e is the number of completed epochs, and e_{total} is the total number of epochs to train the model for. So the time-scheduling we employed is taking a uniformly random time out of a linearly growing interval. However, because the initial condition will have to be enforced too, we take $t = 0$ with probability p . Where p is a hyperparameter of the model, $p = 0.01$ (this is usually enough) unless stated otherwise. Note that: $\hat{P}_\theta(\mathbf{n}, 0) := \mathbb{P}(\mathbf{n}, 0)$, meaning for any state $\mathbf{n}$ the model just returns the initial probability distribution. This is because the initial probability distribution is always known, so there is no need to use the standard pipeline. Thus whenever $t = 0$, the loss becomes equivalent to:

$$\mathcal{L} = D_{KL}(\hat{P}_\theta(\cdot, \delta t) \mid \mathbb{T}\mathbb{P}(\cdot, 0)).$$

After we have selected the time to train at t , we perform a forward pass on all the samples $\mathbf{s}^{(b)}$. As the target is not dependent on θ , we can use eq. (2.18) with this loss function instead of the VAN model loss function. Similar to the VAN model, the resulting gradient will be a REINFORCE-type gradient estimator with reward $\log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) - \log((\mathbb{T}\hat{P}_{\theta^-}(\cdot, t))(\mathbf{s}))$:

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{\mathbf{s} \sim \hat{P}_\theta} \left(\nabla_\theta \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) (\log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) - \log((\mathbb{T}\hat{P}_{\theta^-}(\cdot, t))(\mathbf{s}))) \right). \quad (3.2)$$

We will now dedicate some special attention to the gradient with respect to the output of the MLP. Let $\mathbf{s} \in \mathcal{S}$, where $\mathcal{S}$ is the state space. Then let $\mathbf{a}^{(m)} \in \mathbb{R}^N$ be the post-softmax output of the MLP

of the m 'th run. Similarly let $\mathbf{z}^{(m)} \in \mathbb{R}^N$ be the pre-softmax output of the MLP of the m 'th run, so: $\mathbf{a}^{(m)} = \text{Softmax}(\mathbf{z}^{(m)})$. Note that:

$$\begin{aligned}
& \partial_{a_i^{(m)}} \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) \\
&= \partial_{a_i^{(m)}} \log\left(\prod_{j=1}^M \hat{P}_\theta(s_j \mid s_{<j}, t + \delta t)\right) \\
&= \partial_{a_i^{(m)}} \log\left(\prod_{j=1}^M a_{s_j}^{(j)}\right) \\
&= \partial_{a_i^{(m)}} \sum_{j=1}^M \log(a_{s_j}^{(j)}) \\
&= \sum_{j=1}^M \partial_{a_i^{(m)}} \log(a_{s_j}^{(j)}) \\
&= \begin{cases} 0 & \text{if } s_m \neq i \\ \frac{1}{a_{s_m}^{(m)}} & \text{if } s_m = i \end{cases}.
\end{aligned}$$

Now we can get the gradient with respect to the pre-softmax output of the MLP by left-multiplying the post-softmax derivative by the derivative of softmax in the logits $\mathbf{z}^{(m)}$. The elements of this matrix are given by:

$$\begin{aligned}
& D_{z_j^{(m)}} \text{Softmax}(\mathbf{z}^{(m)})_i \\
&= D_{z_j^{(m)}} \frac{e^{z_i^{(m)}}}{\sum_{k=1}^N e^{z_k^{(m)}}} \\
&= \text{Softmax}(\mathbf{z}^{(m)})_i (\mathbf{1}_{\{i=j\}} - \text{Softmax}(\mathbf{z}^{(m)})_j) \\
&= a_i^{(m)} (\mathbf{1}_{\{i=j\}} - a_j^{(m)}).
\end{aligned}$$

Therefore:

$$\begin{aligned}
& \partial_{z_i^{(m)}} \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) \\
&= \sum_{k=1}^N a_i^{(m)} (\mathbf{1}_{\{i=k\}} - a_k^{(m)}) \partial_{a_i^{(m)}} \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) \\
&= a_i^{(m)} (\mathbf{1}_{\{i=s_m\}} - a_{s_m}^{(m)}) \frac{1}{a_{s_m}^{(m)}} \\
&= \mathbf{1}_{\{i=s_m\}} - a_i^{(m)}.
\end{aligned}$$

So as the loss is dependent on all M runs of the MLP per sample, we get:

$$\nabla_{\mathbf{z}} \mathcal{L} = \mathbb{E}_{\mathbf{s} \sim \hat{P}_\theta} \left(\left(\log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) - \log((\mathbb{T} \hat{P}_\theta(\cdot, t))(\mathbf{s})) \right) \sum_{m=1}^M (\mathbf{1}_{\{i=s_m\}} - a_i^{(m)}) \mathbf{I}_{1 \leq i \leq N} \right).$$

3.1.3 Taking Samples

As mentioned earlier, we take samples from $\hat{P}_\theta(\cdot, t + \delta t)$ during training. The process of sampling is done similarly to doing a single forward pass with a few extra steps. For a sample $\mathbf{s}^{(b)}$, suppose $\mathbf{s}_{<i}^{(b)}$

are already generated for some $1 \leq i \leq M$. Then input the token $x_i := E(i, \mathbf{s}_{<i}^{(b)}, \psi(t + \delta t), \eta_i)$ into the model. We obtain the following conditional probabilities: $P(\mathbf{s}_i^{(b)} = 0 \mid \mathbf{s}_{<i}^{(b)}, t + \delta t), \dots, P(\mathbf{s}_i^{(b)} = N - 1 \mid \mathbf{s}_{<i}^{(b)}, t + \delta t)$, where N is the truncation count for each species. Now let $u \sim U[0, 1]$, and then let a be the smallest integer such that: $\sum_{m=1}^a P(\mathbf{s}_i^{(b)} = m \mid \mathbf{s}_{<i}^{(b)}, t + \delta t) \geq u$. After this we can define: $\mathbf{s}_i^{(b)} := a$, now $\mathbf{s}_{<i+1}^{(b)}$ will be generated. Note that to generate $\mathbf{s}_1^{(b)}$, no prior species count information is needed, therefore this process inductively generates a sample $\mathbf{s}^{(b)} \sim \hat{P}_\theta(\cdot, t + \delta t)$.

3.1.4 Computing $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$

This might seem computationally expensive at first, we will show here that it actually only requires $\mathcal{O}(K)$ multiplications. Recall that: $\mathbb{T} = e^{\delta t \mathbb{W}}$ as found in eq. (2.15), and $\mathbb{T} \approx I + \delta t \mathbb{W}$. Then note that:

$$(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)}) \approx ((I + \delta t \mathbb{W})\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)}) = \hat{P}_\theta(\mathbf{s}^{(b)}, t) + \delta t \sum_{\mathbf{n} \neq \mathbf{s}^{(b)}} [W_{\mathbf{n} \rightarrow \mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{n}, t)] - r_{\mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{s}^{(b)}, t).$$

This is similar to applying one step of the explicit Euler numerical method, where δt is the step size. Because $\hat{P}_\theta(t)$ approximates the actual distribution, it follows that: $\sum_{\mathbf{n} \neq \mathbf{s}^{(b)}} [W_{\mathbf{n} \rightarrow \mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{n}, t)] - r_{\mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{s}^{(b)}, t)$ approximates $\partial_t \mathbb{P}(\mathbf{s}^{(b)}, t)$ as we have seen in eq. (2.12) (the CME).

Then note that: $W_{\mathbf{s}^{(b)} \rightarrow \mathbf{n}} = W_{\mathbf{n} \rightarrow \mathbf{s}^{(b)}} = 0$ for any $\mathbf{n}$ in our state space that is not a *connected* state to $\mathbf{s}^{(b)}$. Meaning that if there is no reaction with stoichiometric column ν such that: $\mathbf{s}^{(b)} = \mathbf{n} + \nu$ or $\mathbf{s}^{(b)} = \mathbf{n} - \nu$, then: $W_{\mathbf{s}^{(b)} \rightarrow \mathbf{n}} = W_{\mathbf{n} \rightarrow \mathbf{s}^{(b)}} = 0$. This shows that $\mathbb{W}$ is actually very sparse for most SRNs, it is the case that:

$$\sum_{\mathbf{n} \neq \mathbf{s}^{(b)}} [W_{\mathbf{n} \rightarrow \mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{n}, t)] - r_{\mathbf{s}^{(b)}} \hat{P}_\theta(\mathbf{s}^{(b)}, t) = \sum_{k=1}^K [a_k(\mathbf{s}^{(b)} - \nu_k) \hat{P}_\theta(\mathbf{s}^{(b)} - \nu_k, t) - a_k(\mathbf{s}^{(b)}) \hat{P}_\theta(\mathbf{s}^{(b)}, t)].$$

This is the same rewrite that has been done between eq. (2.7) and eq. (2.12), now justified better. This shows that calculating an approximation of $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$ actually only requires $\mathcal{O}(K)$ multiplications. Thus we define the target probability p_{target} is an approximation of $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$ by:

$$p_{target} := \hat{P}_\theta(\mathbf{s}^{(b)}, t) + \delta t \sum_{k=1}^K [a_k(\mathbf{s}^{(b)} - \nu_k) \hat{P}_\theta(\mathbf{s}^{(b)} - \nu_k, t) - a_k(\mathbf{s}^{(b)}) \hat{P}_\theta(\mathbf{s}^{(b)}, t)].$$

3.1.5 Selecting a Value for δt

Because the target $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$ is computed by an approximation, it can sometimes occur that it produces a negative probability. Specifically, this can happen whenever δt is too large. Recall from section 3.1.4 that we approximate $(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)})$ by:

$$(\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)}) \approx p_{target} = \hat{P}_\theta(\mathbf{s}^{(b)}, t) + \delta t \sum_{k=1}^K [a_k(\mathbf{s}^{(b)} - \nu_k) \hat{P}_\theta(\mathbf{s}^{(b)} - \nu_k, t) - a_k(\mathbf{s}^{(b)}) \hat{P}_\theta(\mathbf{s}^{(b)}, t)].$$

Then by rearranging these terms we get:

$$p_{target} = (1 - \delta t r_{\mathbf{s}^{(b)}}) \hat{P}_\theta(\mathbf{s}^{(b)}, t) + \delta t \sum_{k=1}^K a_k(\mathbf{s}^{(b)} - \nu_k) \hat{P}_\theta(\mathbf{s}^{(b)} - \nu_k, t).$$

Where $r_{\mathbf{s}^{(b)}}$ is the escape rate away from the state $\mathbf{s}^{(b)}$, as defined in eq. (2.13). Note that propensities are always non-negative, and so are probabilities. Therefore the approximated target probability p_{target}

can only become negative whenever:

$$1 - \delta t r_{\mathbf{s}^{(b)}} < 0 \\ \iff \delta t > \frac{1}{r_{\mathbf{s}^{(b)}}}.$$

Therefore one could pick δt small enough such that this condition always holds. This however, might not always give satisfactory results. If there is some state with a very large escape rate, then selecting δt by this scheme will make δt negligible. In this case the target distribution (at time $t + \delta$) is not meaningfully different from the predicted distribution at time t . Therefore it is sometimes better to set δt to be higher, while risking producing negative target probabilities. When such a negative target probability occurs we use clamping. Meaning that instead of the approximated target probability p_{target} we use:

$$p'_{target} := \max((\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}^{(b)}), \epsilon_2).$$

Where p'_{target} is the approximated and then clamped target probability and ϵ_2 is set to 1^{-10} in the code. Clamping the target probabilities avoids numerical failure for larger values of δt . It allows us to select δt to be large enough that the target distribution (at time $t + \delta$) is meaningfully different from the predicted distribution at time t . The drawbacks however, are that it also changes the target distribution and might introduce bias. Therefore, selecting a value for δt involves a trade-off between the stability of the target approximation and making the learning signal sufficiently informative.

3.1.6 Baseline

We use an exponentially moving average (EMA) baseline to reduce gradient variance. Where the baseline we use for every epoch is defined inductively by:

$$b_1 := 0 \\ b_{k+1} := \alpha b_k + (1 - \alpha)L_k.$$

Where b_k is the baseline for epoch k , L_k is the loss of epoch k , and we have taken α to be 0.95. The usual method that is employed is taking the baseline as the loss of the same epoch. This method makes it so that there is no need to calculate an entire batch to get the baseline, we can already calculate it at the beginning of every epoch. Meaning we do not have to store all cached values belonging to every single run within an epoch, or do the forward pass twice for every sample within an epoch.

Now define $R(\mathbf{s}) := \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) - \log((\mathbb{T}\hat{P}_\theta(\cdot, t))(\mathbf{s}))$. Note that $R(\mathbf{s})$ is just the reward associated with the sample $\mathbf{s}$. Then we correct the gradient found in eq. (3.2) for epoch k with the baseline b_k as follows:

$$\nabla_\theta \mathcal{L} = \mathbb{E}_{\mathbf{s} \sim \hat{P}_\theta} \left(\nabla_\theta \log(\hat{P}_\theta(\mathbf{s}, t + \delta t)) (R(\mathbf{s}) - b_k) \right).$$

Note that if we used the usual method, where we set the baseline to be the loss of that epoch, that the expected value would remain the same. The only difference would be that the variance has been reduced when approximating the gradient.

3.2 Time-Conditioned Model with Masked Attention Mechanism

For this model we use the same general method as for the basic time-conditioned model. The difference is that we first pass the tokens into a masked attention mechanism and then pass the output of that into the MLP as the input. The rationale behind adding the masked attention mechanism was to improve conditioning of species on each other. Only the forward pass is described again for this approach. Every other detail from section 3.1 can be applied to this model as well. Note that the attention mechanism was poorly tested, and none of the results that could be considered successful in section 4 have used it. This section should be seen as a possible extension of the model proposed in section 3.1.

3.2.1 Masked Attention

In a famous paper from 2017 [3], scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) := \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{n \times d_k}.$$

Where: $Q := XW_Q$, $K := XW_K$ and $V := XW_V$ are the query, key and value matrices respectively. The weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are tunable parameters, $X \in \mathbb{R}^{n \times d}$ serves as the input matrix. Furthermore, $\frac{QK^\top}{\sqrt{d_k}}$ is the score matrix. Note that:

$$(QK^\top)_{ij} = \langle q_i, k_j \rangle.$$

Where $q_i = W_Q^\top x_i$ and $k_j = W_K^\top x_j$, $x_i \in \mathbb{R}^d$ is the i 'th input vector (the transposed i 'th row of X). Furthermore, $\langle \cdot, \cdot \rangle$ is the standard euclidean inner product or "dot-product". The dot-product is there to give a sense of how similar the query q_i and the key k_j are. It returns the largest value when q_i and k_j are equal, it returns the smallest value when q_i and k_j point in opposite directions ($q_i = -k_j$). In general, the larger $\langle q_i, k_j \rangle$, the more similar the directions which q_i and k_j point in are. Whenever $\langle q_i, k_j \rangle$ is a relatively large number, we say that x_j *attends* to x_i . In other words this means that the token x_j provides useful context to the token x_i , it is relevant to x_i .³

Softmax is applied row-wise. The scaling by $\frac{1}{\sqrt{d_k}}$ of $QK^\top$ for the dimension $d_k \in \mathbb{N}$ is done to counteract the potentially large magnitudes of the dot-products (which scale with d_k). This can push the softmax function into regions where it has extremely small gradients, which would be a problem during training. The purpose of softmax is simply to ensure the values in the rows are between 0 and 1 and add up to 1. Recall that the score matrix provides us with a sense of how relevant tokens are to each other. Where positive values of $\langle q_i, k_j \rangle$ indicated more relevance of the token x_j to x_i . After applying softmax, the values in the rows of the matrix now read the weighted relevance for each token.

In the context of our application we do not want x_i to attend to any input token x_j , where $j > i$. This is because we want the model to predict the conditional probability $\hat{P}_\theta(n_i \mid n_{<i}, t)$ on input x_i . Whenever $j > i$, x_j would contain information about n_i , which should not be available when evaluating x_i . To ensure this, we mask the score matrix by adding a matrix $M \in \mathbb{R}^{n \times n}$. Where M is a matrix with all values in the upper triangle set to $-\infty$ and the rest set to 0:

$$\text{MaskedAttention}(Q, K, V) := \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \in \mathbb{R}^{n \times d_k}. \quad (3.3)$$

Adding M from the score matrix ensures that the upper triangle of the post-softmax score matrix is set to 0. Meaning that x_i can not attend to any x_j for which $j > i$.

3.2.2 Forward Pass

The input tokens are defined the same way as in eq. (3.1). But instead of being fed directly into the MLP, we will first pass it through a masked attention mechanism as defined in eq. (3.3). For each $1 \leq i \leq M$, define: $h_i := (\text{MaskedAttention}(Q, K, V))_i^\top \in \mathbb{R}^{d_k}$. Where the subscript i refers to the i 'th row. Typically we set $d_k < d_x$. For each $1 \leq i \leq M$, the MLP produces the logits $\hat{P}_\theta(n_i = m \mid n_{<i}, t)$ for all $0 \leq m < N$ on input h_i . After processing x_i for all i we then produce: $\hat{P}_\theta(n, t) = \prod_{i=1}^M \hat{P}_\theta(n_i \mid n_{<i}, t)$. The following is a simple diagram explaining the employed machine learning pipeline for the time-conditioned model with a masked attention mechanism:

$$(i, n_{<i}, \psi(t), \eta_i) \rightarrow x_i \rightarrow \text{masked attention block} \rightarrow h_i \rightarrow \text{MLP} \rightarrow \text{softmax} \rightarrow \hat{P}_\theta(n_i \mid n_{<i}, t).$$

³Note that in our implementation of the attention mechanism, the data matrix X is transposed. So the whole

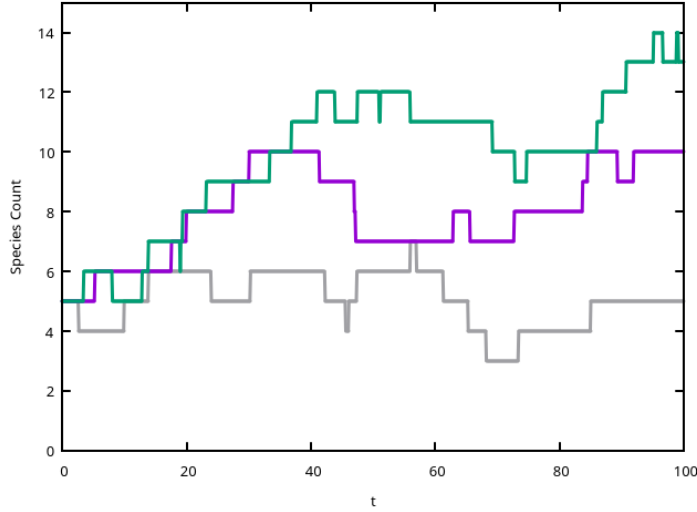


Figure 1: Three different paths, all simulations of the species count over time of the birth-death process. Where the initial distribution is the delta distribution with the species count set to 5. The reaction rates are $k_1 = 0.1$ and $k_2 = 0.01$. See section 4.1 for the definition of the birth-death process.

4 Numerical Experiments and Results

To see how the model performs we have done some tests that are similar to some tests that were done in the VAN paper [2]. These are the tests on the birth-death process, gene expression SRN and the linear signaling cascade. Like in the VAN paper, we benchmarked all the results against another numerical method. Specifically we used the Gillespie algorithm [1] to simulate paths of species counts over time. An example of one of such paths of species counts over time is shown in fig. 1. Specifically for the species count over time of the single species in the birth-death process, see section 4.1 for a definition. To compute the joint probability distributions at some time, we count for every state the amount of paths that were in that state at that time. Then we divide every one of those counts by the amount of paths we simulated to obtain the joint probability distribution at that time. Similarly, the mean and the standard deviations at some time are computed with samples taken from these paths at that time. Another measure that we used to compare our results with what the Gillespie algorithm produces is the Hellinger distance. It is just a way to measure how similar two probability distributions are. Let $P := (p_1, \dots, p_k)$ and $Q := (q_1, \dots, q_k)$ be two discrete probability distributions, the Hellinger distance between P and Q is defined as:

$$H(P, Q) := \frac{1}{\sqrt{2}} \sqrt{\sum_{i=1}^k (\sqrt{p_i} - \sqrt{q_i})^2}.$$

Note that when taking the Hellinger distance, we always compare a joint probability distribution produced by the trained time-conditioned model, to one produced using the Gillespie algorithm⁴. We have also done some tests that are specific to this model, like the test on the time embedding. The hyperparameters that we used in every test can be found in table 1, in addition to the parameter count and the training time. Note that if a hyperparameter does not appear in this table that it was set to the default value as specified in section 3. All the plots were generated using gnuplot [5].

attention mechanism is transposed from the one we show here. This is to make it align better with the column-wise ordering of matrices we use.

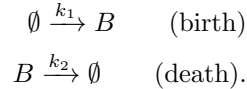
⁴Recall that we truncate the state space, so both of these produced probability distributions are indeed discrete.

Test	N	T	δt	Epochs	d_h	d_t	d_k	$\dim(\theta)$	Training time
Birth-death process (4.1)	30	100	0.01	500000	16	1	N/A	657	0.89
Gene expression (4.2)	100	3600	0.1	300000	64	32	32	13124	13.27
Signaling cascade (4.3)	20	10	0.01	300000	64	32	N/A	4404	5.17
Time embedding (a) (4.4)	15	1	0.01	100000	32	1	N/A	1007	0.56
Time embedding (b) (4.4)	15	1	0.01	100000	32	16	N/A	1487	0.71
Time embedding (c) (4.4)	15	1	0.01	100000	32	32	N/A	1999	0.89
Time embedding (d) (4.4)	15	1	0.01	100000	32	32	N/A	2095	0.85
Runtime scaling (4.5)	20	10	0.01	100000	64	32	N/A	variable	variable

Table 1: Some hyperparameter settings used in each test, in addition to the parameter count and the training time. If a hyperparameter does not appear in this table then it is set to a default value mentioned in section 3. Whenever N/A appears in the d_k column that means that no attention mechanism was used, like in section 3.1. The training time is in hours, all computations were done on a single Ryzen 5 3600 CPU core.

4.1 The Birth-Death Process

The birth-death process is a relatively simple SRN. There are only two reactions and one species. In the VAN paper they also tested on this SRN. It is therefore a good starting point to see whether this approach works at all. The general description goes as follows:



Where B is our species. Our results training the model on the birth-death process are shown in fig. 2. The initial probability distribution is the delta distribution⁵ with 5 of the species B . For the tests we used the reaction rates: $k_1 = 0.1$ and $k_2 = 0.01$. Some of the used parameters of the model can be found in table 1, the other parameters are set to default values as specified in section 3. The results shown in fig. 2 are promising. The model is able to fit the entire joint probability distribution up to the time T with little error. The species count mean and standard deviation over time are similar to those produced using the Gillespie algorithm at all sampled points in time. Furthermore, the Hellinger distance to the joint probability distribution produced using the Gillespie algorithm is low and flat with respect to time.

⁵We define the delta distribution as: $\mathbb{P}(\mathbf{n}) = \begin{cases} 1 & \text{if } \mathbf{n} = \mathbf{n}_0 \\ 0 & \text{if } \mathbf{n} \neq \mathbf{n}_0 \end{cases}$. For some $\mathbf{n}_0$ in the state space.

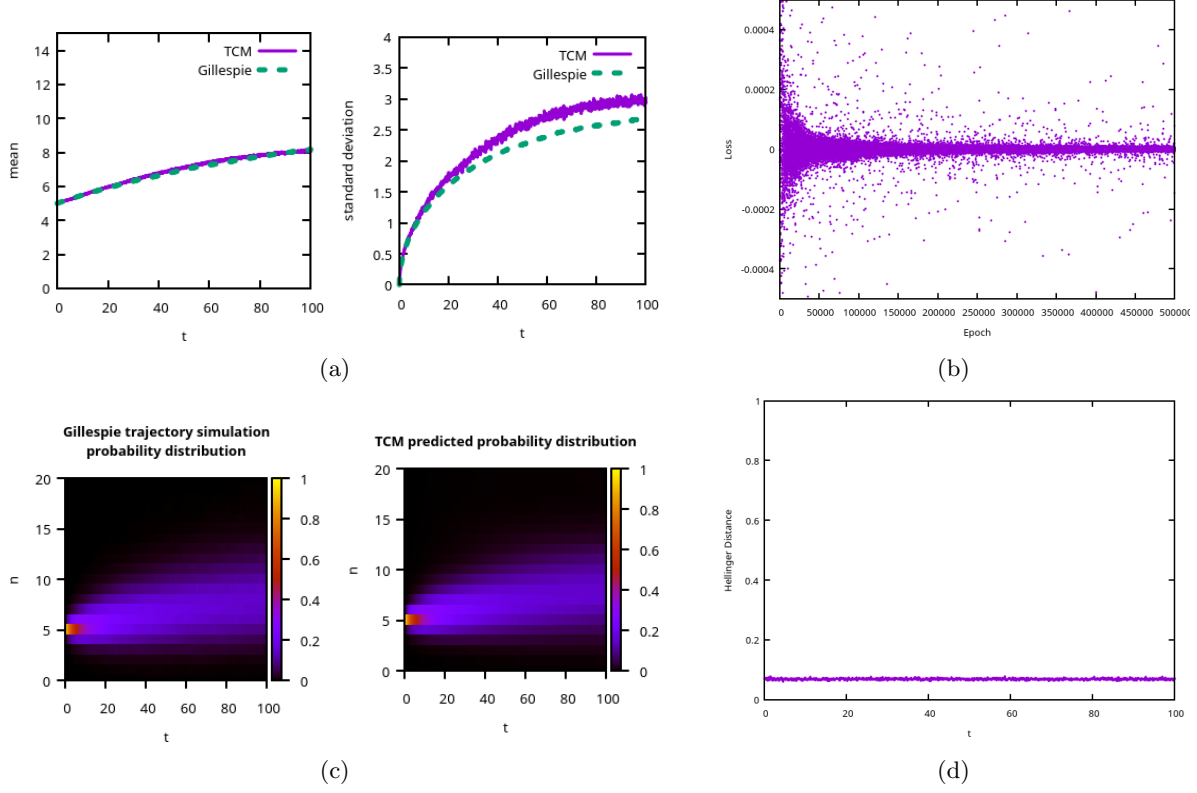
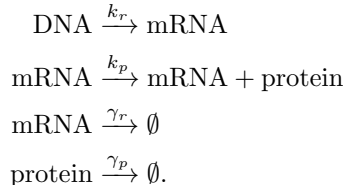


Figure 2: The results of the birth-death process test. (a) The mean and the standard deviation of the birth-death process over time. The time t is on the horizontal axis and the species count mean (left plot) and standard deviation (right plot) is on the vertical axis. The dotted green line shows the mean and standard deviation produced using the Gillespie algorithm. The solid purple line shows the mean and standard deviation produced by the trained time-conditioned model. (b) The training loss for every epoch training the birth-death process. (c) The full probability distribution of the birth-death process. The time t is on the horizontal axis and the species count n is on the vertical axis. The color spectrum shows the predicted probability $\mathbb{P}(n, t)$ at that point. The plot on the left was produced using the Gillespie algorithm. The plot on the right was produced by the trained time-conditioned model. (d) The Hellinger distance over time between the full probability distributions produced by the time-conditioned model and one produced using the Gillespie algorithm.

4.2 Gene Expression

The gene expression SRN is used to model the randomness of fundamental cellular processes like transcription and translation. The species are mRNA and protein, there are four reactions defined as follows:



Where DNA is not taken into the SRN as a species, it is only included to more accurately describe the process of transcription in the first reaction. For the test we used the delta distribution with both species counts set to 0 as the initial probability distribution. Furthermore, we used the following reaction rates: $k_r = k_p = \gamma_r = 0.1$ and $\gamma_p = 0.002$. The hyperparameters that we used can be found in table 1. For the test we have done the Hellinger distance was close to 1 at every tested point in time⁶, which shows a complete failure to learn any distribution correctly, see fig. 3(c). This is further corroborated by the means and the standard deviations of the learned distributions over time, which do not correspond to the ones produced using the Gillespie algorithm, as seen in fig. 3(a). Despite this, fig. 3(b) does show the training loss decreasing with the number of epochs. These results suggest that, for this parameter setting, the trained model may have collapsed to a poor approximation that does not sufficiently use the time and state inputs. The output of the MLP seems to return almost constant conditional probabilities with respect to the input. This interpretation is corroborated by the fact that the mean and standard deviation of the mRNA and protein counts are almost the same at all points in time, as shows in fig. 3(a). There are a few possible causes that might have caused this total failure. For instance, the model architecture might have simply been inadequate for this test. Another possibility is that the time-scheduling is inadequate; the sequence of gradient signals from different times that the model receives during training makes it fail. It could also be the case that the target approximation error is too high; δt is considerably larger than the value we used in other tests, see section 3.1.5. Another cause could be that there is inadequate treatment of the species scales; the protein count is on average much higher than the mRNA count. This can be seen by the means that were produced using the Gillespie algorithm in fig. 3(a) (the two dotted lines, left plot).

⁶Note that the maximum value that the Hellinger distance can take is 1.

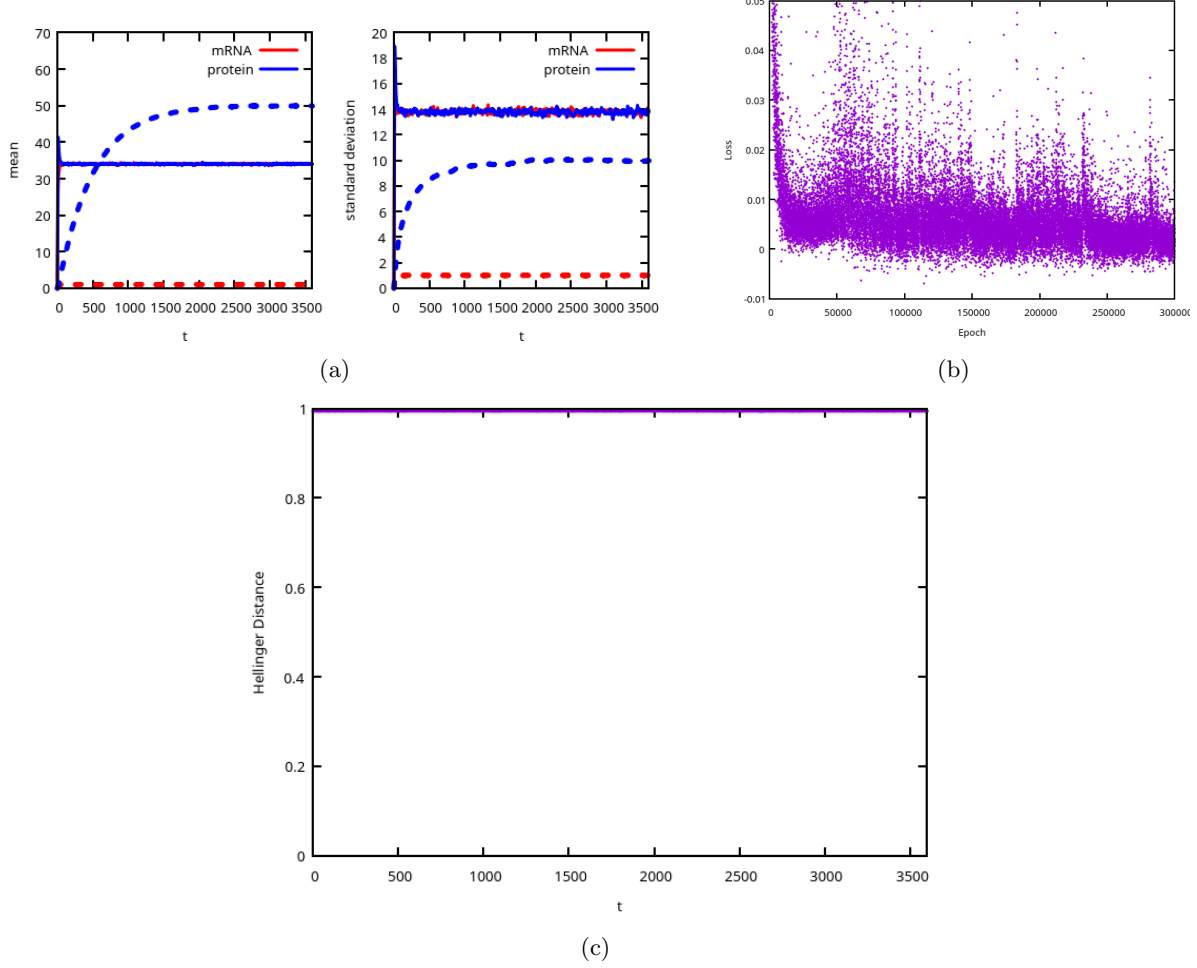
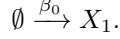


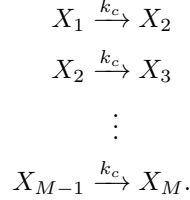
Figure 3: The results of the gene expression test. (a) The mean and the standard deviation of the gene expression SRN over time. The time t is on the horizontal axis and the species counts means (left plot) and standard deviations (right plot) is on the vertical axis. The dotted lines show the means and standard deviations over time produced using the Gillespie algorithm. The solid lines show the means and standard deviations over time produced by the trained time-conditioned model. The different colors denote the two species of the gene expression SRN; blue is for protein, red is for mRNA. (b) The training loss for every epoch training the gene expression SRN. (c) The Hellinger distance over time between the full probability distributions produced by the time-conditioned model and one produced using the Gillespie algorithm.

4.3 Linear Signaling Cascade

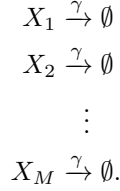
The linear signaling cascade SRN will mostly serve as a scalable benchmark for our model. In general there are M species $X_1, \dots, X_M$ and $2M$ reactions. One reaction is the synthesis reaction for X_1 :



Then there are the catalytic reactions:



Finally, all species also have decay reactions with decay rate γ :



The results training the model on the linear signaling cascade with $M = 2$ are shown in fig. 4. The initial distribution is the delta distribution with both species counts set to 0. The reaction rates that we used were: $\beta_0 = 10$, $k_c = 5$ and $\gamma = 1$. The hyperparameters that we used can be found in table 1. The time-conditioned model was able to fit the entire probability distribution up to the time T decently well. The means and standard deviations of both species counts produced by the time-conditioned model closely align with that produced using the Gillespie algorithm at all times, as shown in fig. 4(a). In fig. 4(c) it is also shown that the joint probability distributions produced by the time-conditioned model and that produced using the Gillespie algorithm at $t = 1$ and $t = 10$ look very similar. While fig. 4(d) shows that they are very similar at all times, having a low Hellinger distance that is also flat with respect to time. It is however important to note that this particular test was only done with $M = 2$. The signaling cascade is scalable, but the results shown in this section do not necessarily imply that the model would perform well for the signaling cascade with $M > 2$.

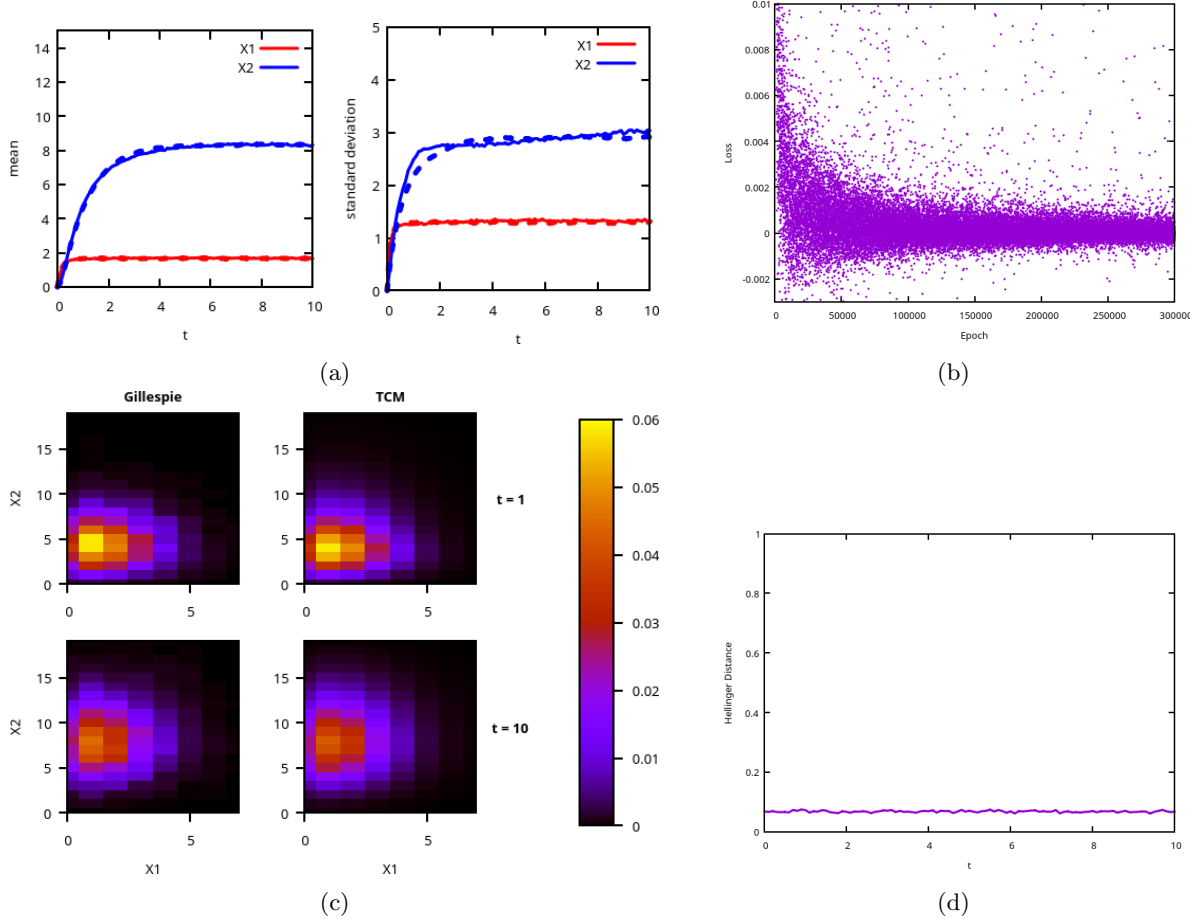


Figure 4: The results of the signaling cascade test. (a) The mean and the standard deviation of the signaling cascade test over time. The time t is on the horizontal axis and the species count mean (left plot) and standard deviation (right plot) is on the vertical axis. The dotted lines shows the mean and standard deviation produced using the Gillespie algorithm. The solid lines shows the mean and standard deviation produced by the trained time-conditioned model. (b) The training loss for every epoch training the signaling cascade. (c) The full probability distribution of the signaling cascade test at time $t = 1$ (upper two plots) and $t = 10$ (lower two plots). The count of the first species X_1 is on the horizontal axis and the count of the second species X_2 is on the vertical axis. The color spectrum shows the predicted probability $\mathbb{P}(n_1, n_2, t)$ at that point. The two plots on the left were produced using the Gillespie algorithm. The two plots on the right were produced using the trained time-conditioned model. (d) The Hellinger distance over time between the full probability distributions produced by the time-conditioned model and one produced using the Gillespie algorithm.

4.4 Time Embedding Tests

We ran some tests using different time-embeddings ψ on the linear signaling cascade SRN with $M = 2$. One test was done with no time embedding at all, instead passing in the time t linearly without transforming it. Then two tests were done using a sinusoidal transformation, one with $d_t = 16$ and the other with $d_t = 32$. Specifically, the sinusoidal time embedding used was:

$$\psi(t) = (\sin(\frac{2\pi t}{T}), \sin(\frac{4\pi t}{T}), \dots, \sin(\frac{2d_t\pi t}{T}))^\top. \quad (4.1)$$

The last test was done with a learnable embedding map, where $d_t = 32$. This is just the default time embedding map that we describe in section 3.1.1. For all the tests the initial probability distribution is the delta distribution with both species counts set to 0. The reaction rates that we used were: $\beta_0 = 10$, $k_c = 5$ and $\gamma = 1$. The hyperparameters that we used can be found in table 1. The test results can be seen in fig. 5. The test results show that a proper time-embedding is essential for the model to learn the entire distribution over time, a simple linear map will not suffice for all SRNs. The linear map produces an almost constant mean and standard deviation at all sampled times, as shown by fig. 5(a). This likely means that the time is almost ignored as an input. The sinusoidal embeddings perform a lot better, the time input is clearly being used, see fig. 5(b) and fig. 5(c). The means are very close to the control at all times. the standard deviations are too high at some points in time for the second species X_2 , but overall seems to approach the control. The learnable time embedding clearly performs the best, see fig. 5(d). The mean over time of both species are very similar to those of the control. The standard deviation over time is also very close to the control for both species. It is also noticeably less noisy than with the sinusoidal time embeddings.

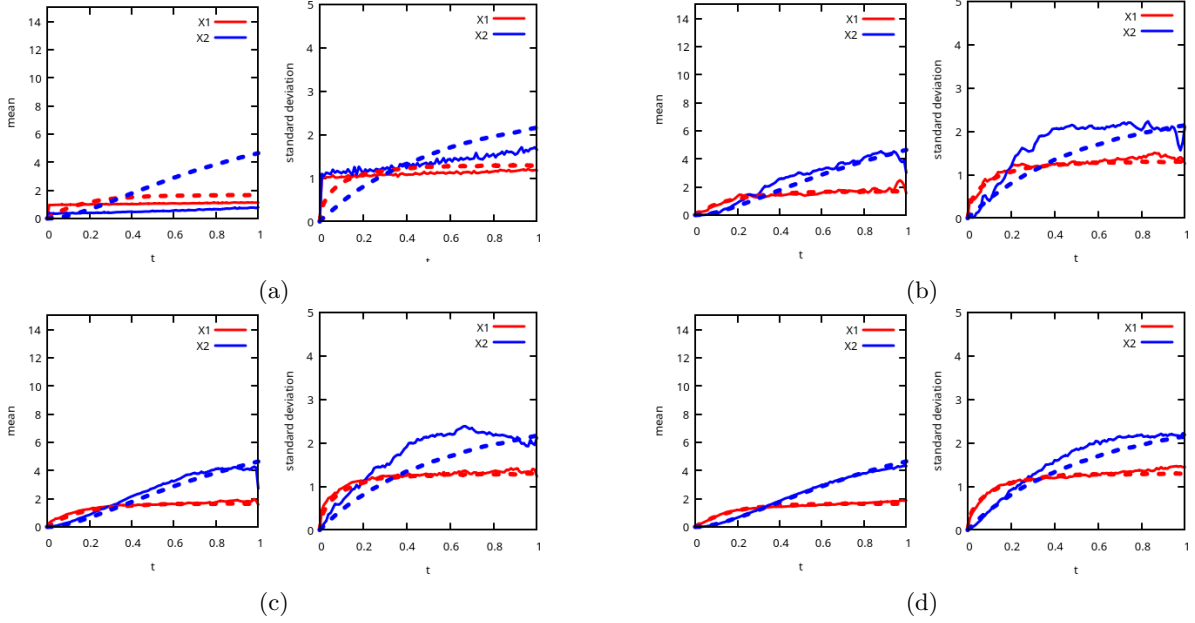


Figure 5: The mean and the standard deviations of four different time embedding tests. For every plot the time t is on the horizontal axis and the species count mean (left plot) and standard deviation (right plot) is on the vertical axis. The dotted lines shows the mean and standard deviation produced using the Gillespie algorithm. The solid lines shows the mean and standard deviation produced by the trained time-conditioned model. (a) Results for the linear test, no time embedding was used. Instead the time t was passed without transforming it. (b) Results with the sinusoidal time embedding where $d_t = 16$, as defined in eq. (4.1). (c) Results with the sinusoidal time embedding where $d_t = 32$, as defined in eq. (4.1). (d) Results with a learnable time embedding where $d_t = 32$, described in section 3.1.1.

4.5 Runtime Scaling

We have done some tests to determine how the computation time for training scales with respect to the amount of species in the SRN. For this we used the linear signaling cascade to benchmark, as defined in section 4.3. The linear signaling cascade is easily scalable, and so it is a natural choice to benchmark with. The hyperparameters that we used can be found in table 1. The initial distribution is the delta distribution with both species counts set to 0. The reaction rates that we used were: $\beta_0 = 10$, $k_c = 5$ and $\gamma = 1$. All tests were performed on a single Ryzen 5 3600 CPU core. In the current implementation and hardware setting, the training time grows rapidly with the number of species, see fig. 6. In theory the scaling should be similar to what they found with the VAN, which can be seen in figure 3f of the VAN paper [2]. Although it is important to note that they took advantage of GPU architecture to get these results, which scale well. Therefore, before drawing a definitive conclusion about the intrinsic scaling of the time-conditioned method, the implementation needs to be optimized further. In particular, the implementation could take advantage of GPU batching.

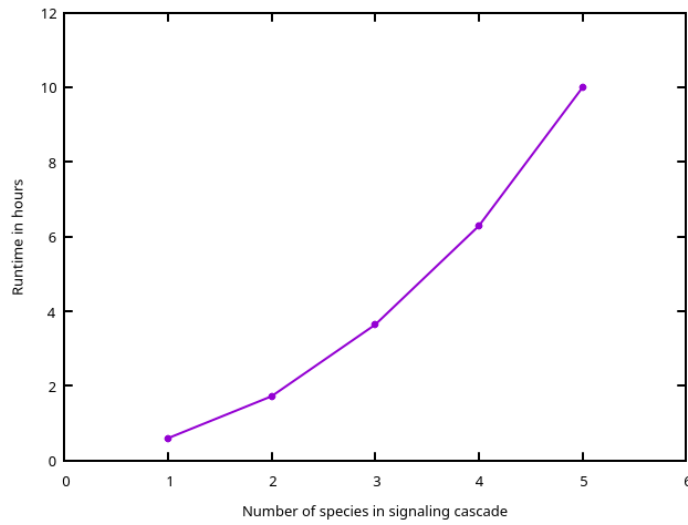


Figure 6: Results of the training computation time tests. All tests were done on the linear signaling cascade, with differing species counts (horizontal axis). The runtime in hours is on the vertical axis.

5 Discussion

The time-conditioned model that we used to solve stochastic reaction networks numerically shows promise. It has a few key strengths that set it apart from the VAN approach. For example, when the model successfully trains a SRN, it learns a shared time-conditioned representation of the joint probability distribution. This means that, unlike with the VAN approach, the model can be evaluated in any time. Also, the model performs well on certain simple SRNs, like the birth-death process and the signaling cascade for small M . Not only does it solve these SRNs with little error, it also needs very few parameters to do so. The training computation times are also manageable for these SRNs. On the other hand, the developed method is still very flawed and possesses some key limitations. One of these is that the training computation time scales rapidly with M in the current implementation, as shown in section 4.5. Although this might be solved by implementing more efficient algorithms (all code was custom-made) or by taking advantage of the GPU. Another limitation is that the model is sensitive to the choice of a time embedding, which was shown in section 4.4. The time-conditioned model also inherits a limitation from the VAN approach; it is sensitive to the choice of δt , as discussed in section 3.1.5. Lastly, the model still fails to learn the gene expression SRN correctly with the parameters we tried, as shown in section 4.2. This could have to do with a multitude of different factors, some of which we will now discuss as a possible direction for future research. The time-scheduling is an obvious avenue for further research, as it was barely experimented on. Specifically, one might work on an adaptive time-scheduling. Where regions in $[0, T]$ where more error is observed are drawn from more often to train on. This might make the model converge faster to the solution of the CME. Then there is also the embedding map E , which is not properly implemented yet, and had thus just been the identity map for all tests. A proper embedding map to start out with could just be a two-layer MLP. Another possible avenue for further research could be testing the effectiveness of the attention mechanism. Furthermore, one could explicitly give more structural information to the model, like preservation laws of the SRN. Lastly, the model is very parallelizable, but this was not taken advantage of in our tests. All of our tests were done on a single CPU core, thus the training time is a good indication of how many computations were needed for training. It is however not a good indication of the training time when computing in parallel. One could take advantage of parallel computing by speeding up affine transformations, which GPUs are very good at. To do this more effectively one could adapt the code to be able to do the computations for one epoch in a few big affine transformations. Another obvious avenue for further research is testing the model on higher-dimensional SRNs, both in terms of the reaction count K and the amount of species M .

6 Conclusion

In this thesis we proposed a globally time-conditioned autoregressive network model for solving SRNs. It is a model that approximates the joint probability distribution of the state space over time of a SRN. We contrasted it with a previous method for doing this that trained a VAN model for every time step of a discretized time interval. We also investigated our approach numerically and found that our model managed to approximate the joint probability distribution with little error for some simpler SRNs, like the birth-death process and the linear signaling cascade. In these cases it also required less time to train than the VAN, using fewer parameters. However, for the gene expression SRN, the model has been found to completely fail. Furthermore, the training time has been found to not scale well with the amount of species in the SRN. The results of our proposed method are promising, although they also show that the model is still very flawed.

7 Code Availability

A from-scratch custom implementation of the time-conditioned model as described in section 3 can be found on <https://github.com/YT41/bachelorScriptie/tree/main>.

References

- [1] Daniel T Gillespie. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of computational physics*, 22(4):403–434, 1976.
- [2] Ying Tang, Jiayu Weng, and Pan Zhang. Neural-network solutions to stochastic reaction networks. *Nature Machine Intelligence*, 5(4):376–385, 2023.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [4] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- [5] Thomas Williams and Colin Kelley. gnuplot, 2013.